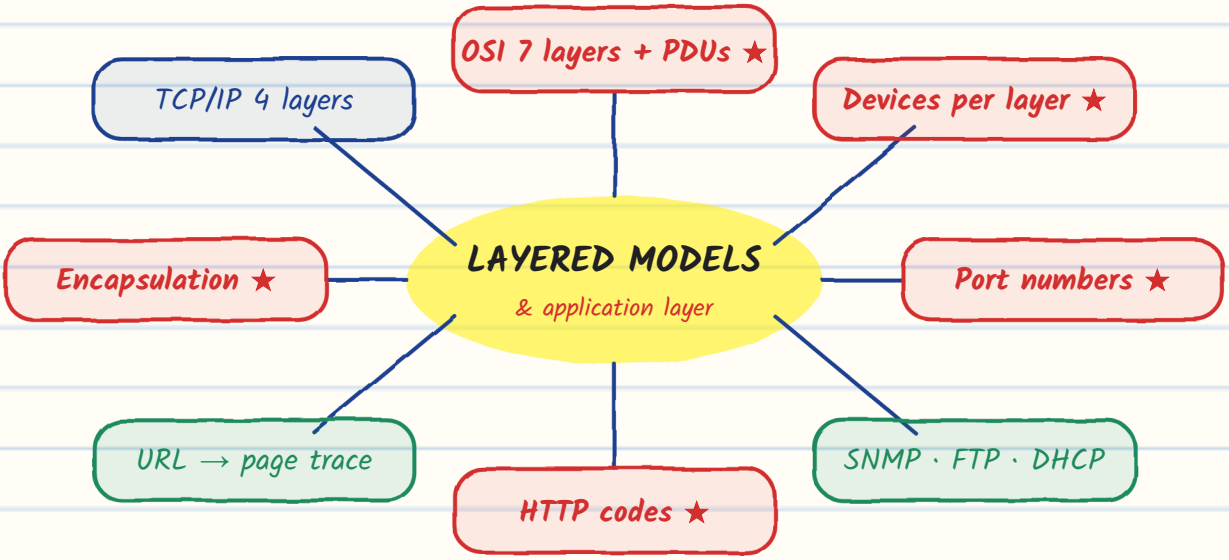
OSI Model, HTTP, WWW & SNMP

7 layers · TCP/IP · encapsulation · URL trace · HTTP · ports · SNMP: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = applied / walk-through

➜ Syllabus coverage

"Computer Networking and Security: Protocol hierarchies ... Domain Name System ... Email and its privacy; SNMP; HTTP; World Wide Web." This notebook covers the OSI and TCP/IP layer models, encapsulation, the end-to-end journey of a web request, HTTP, the Web, SNMP and the well-known ports. Protocol internals live in the neighbouring notes: data link & LANs, routing, IP/ARP, TCP/UDP, DNS & email.

High-yield list

① 7 layers in order
② PDU: bit, frame, packet, segment
③ hub L1 · switch L2 · router L3
④ HTTP 80 · HTTPS 443
⑤ status-code classes
⑥ SNMP UDP 161 / 162

Legend yellow = key term · red circle = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

➜ Contents

✓ Layers & the OSI model ..... p. 2
✓ Layer duties, addresses, devices ... p. 3
✓ TCP/IP model, encapsulation ... p. 4
✓ Headers; trace: opening a URL ... p. 5
✓ HTTP & HTTPS ..... p. 6
✓ WWW, URL, FTP, Telnet, SSH ..... p. 7
✓ DHCP & well-known ports ..... p. 8
✓ SNMP ..... p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

1 Layers & the OSI Reference Model high-yield

Protocol hierarchy Network software is split into layers; each layer gives a **service** to the layer above through an **interface** and talks to its **peer** layer on the other host by a **protocol**. Only layer 1 really moves bits; the others talk to their peers *virtually*. A layer can change (Wi-Fi ↔ Ethernet) without touching the rest.

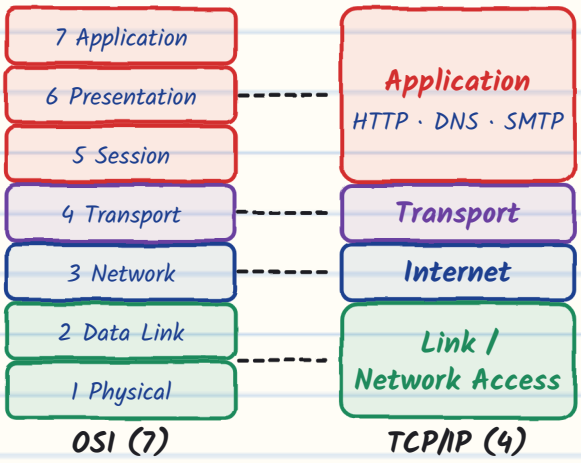
OSI = Open Systems Interconnection, by ISO (ISO 7498, 1984; also ITU-T X.200). **7 layers**; a reference model: its own protocols never caught on, but its vocabulary is used everywhere.

#	Layer	Main job (one line)	PDU	Examples
7	**Application**	network services for the user/app	data	HTTP, FTP, SMTP, DNS, SNMP
6	**Presentation**	translation, **encryption**, **compression**	data	JPEG, MPEG, ASCII, SSL/TLS*
5	**Session**	dialog control, **checkpoints**	data	NetBIOS, RPC, PPTP
4	**Transport**	**process-to-process**: ports, reliability	**segment**	TCP, UDP
3	**Network**	**host-to-host**: IP address, **routing**	**packet**	IP, ICMP, IGMP, OSPF
2	**Data Link**	**node-to-node** (hop): framing, MAC, CRC	**frame**	Ethernet, PPP, HDLC, Wi-Fi
1	**Physical**	raw **bits**: voltage, pins, medium	**bit**	RJ-45, fibre, RS-232, DSL

* SSL/TLS: see the exam trap on the next sheet. UDP's PDU is called a **user datagram**. Layer numbers start at the **bottom**: Physical = 1.

Memory trick 7 → 1: "**A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing".
1 → 7: "**P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way".

PDU trick L4 → L1: "**S**ome **P**eople **F**ear **B**irthdays" = Segment, Packet, Frame, Bit.



Exam trap Layers 4–7 are **end-to-end**: they exist only in the two hosts. A router implements only layers **1–3**, a switch 1–2, a hub 1. "Layer 3 device" = router; "Layer 2 device" = switch.

2 Who Does What: Key Duties per Layer

MCQ favourite

Duty asked in the MCQ	Layer
bit synchronisation, line coding, topology, data rate	1 Physical
framing, **physical (MAC) addressing**, hop-to-hop error & flow control, media access (CSMA/CD)	2 Data Link
logical (IP) addressing, routing, fragmentation, TTL	3 Network
port addressing, segmentation & reassembly, end-to-end error/flow control, congestion control	4 Transport
dialog control (half/full duplex), checkpoints, login session	5 Session
encryption/decryption, compression, character code translation (ASCII ↔ EBCDIC)	6 Presentation
file transfer, mail, web, remote login, directory services	7 Application

→ Three addresses, three layers

↙ flow & error control appear twice: L2 (per hop) and L4 (end to end)

Address	Layer	Size	Scope / example
MAC (physical)	2	48 bit	one link · 3C-52-82-1A-0F-9B
IP (logical)	3	32 / 128 bit	end to end · 192.168.1.10
Port	4	16 bit	one process · 0–65535 · :443

→ Devices by layer

Layer 1	Layer 2	Layer 3	Layer 4–7
hub · repeater cable, modem	switch · bridge NIC, access point	router layer-3 switch	gateway firewall, proxy
1 collision domain	splits collision domains	splits broadcast domains	translates protocols

reads: bits → MAC → IP → ports / content

Exam trap: borderline protocols **ARP** maps IP → MAC: most MCQ keys put it in the **Network layer**, though it really works between L2 and L3 (TCP/IP puts it in the link layer). **SSL/TLS**: textbook MCQs say **Presentation** (encryption); in practice it runs on top of TCP, below HTTP. **NIC**: L1 + L2. Pick the answer the option list allows.

IT link Troubleshoot bottom-up: link light / cable (L1) → arp -a, switch port (L2) → ping, tracert (L3) → telnet host 443 or Test-NetConnection (L4) → browser / curl (L7).

3 The TCP/IP (Internet) Model

high-yield

Definition The model the Internet actually runs on, from the US DoD/ARPANET work (Cerf & Kahn, 1974); ARPANET switched to TCP/IP on **1 Jan 1983**. RFC 1122 defines **4 layers**: **Application** (HTTP, DNS, SMTP, DHCP, SNMP) · **Transport** (TCP, UDP) · **Internet** (IP, ICMP, IGMP) · **Link** (Ethernet, Wi-Fi, PPP, ARP). Textbooks often use a 5-layer hybrid: Link split into Data Link + Physical.

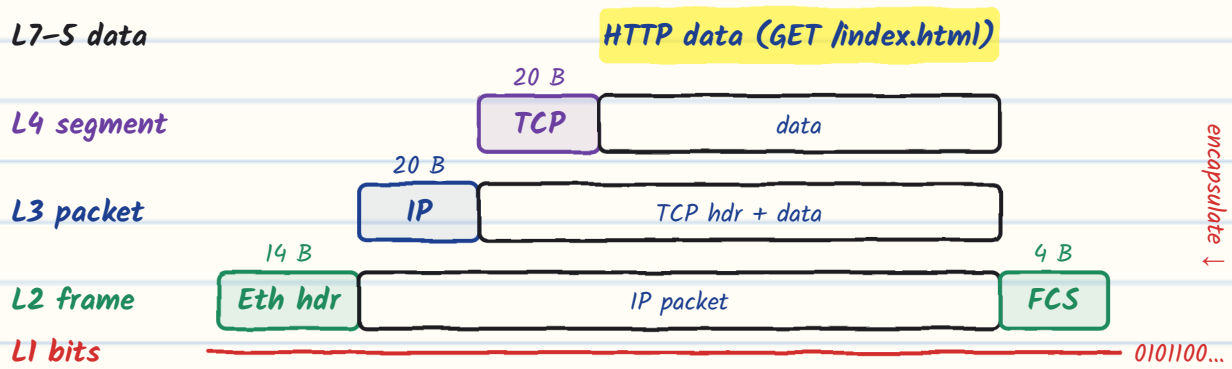
Point	OSI	TCP/IP
Layers	7	4 (5 in the hybrid)
Origin	ISO: model first, protocols later	DoD: protocols first, model later
Service / interface / protocol	clearly separated	not clearly separated
Network layer	connectionless + connection-oriented	**connectionless only** (IP)
Transport layer	connection-oriented only	both (TCP and UDP)
Session, Presentation	separate layers	folded into Application

Memory trick "**3-1-1-2**": OSI's top **3** → TCP/IP Application, **1** → Transport, **1** → Internet, bottom **2** → Link. So OSI **Network** = TCP/IP **Internet**; "Network Access" = OSI L1 + L2, not L3.

4 Encapsulation & Decapsulation

written favourite

Definition Going **down**, each layer treats what it gets as its **payload (SDU)** and adds its own **header** (the data link layer also adds a **trailer**) to form its PDU. Going **up** at the receiver, each layer reads and strips its own header: **decapsulation**.



headers go on the front; only layer 2 adds a trailer (FCS = CRC-32)

Header	Min. size	Key fields
TCP	20 B	source/destination port, sequence & ack no., flags, window
UDP	8 B	source/destination port, length, checksum
IPv4	20 B	source/destination IP, TTL, protocol (6 = TCP, 17 = UDP)
Ethernet II	14 + 4 B	destination/source MAC, EtherType (0x0800 = IPv4), FCS

Worked example ✍ Ethernet MTU = 1500 B (the IP packet). TCP payload per frame = 1500 − 20 − 20 = **1460 B** (the usual MSS). Frame = 1460 + 20 + 20 + 14 + 4 = 1518 B → efficiency 1460/1518 ≈ **96.2%**.

Exam trap At every **router** the frame is stripped and a new one built: **MAC addresses change each hop**, the **IP addresses stay the same** end to end (unless NAT), and TTL drops by 1. Russian dolls: each layer opens **only its own** doll.

5 What Happens When You Open a URL

written favourite

You type https://www.example.com/index.html. The PC already has its IP, mask, gateway and DNS server from **DHCP**.

1. **Parse URL**: scheme https → port **443**; host www.example.com; path /index.html. Check browser and OS caches.
2. **DNS** (UDP 53): ask the resolver for the A/AAAA record → the server's IP. (details: DNS notes)
3. **ARP**: the server is on another network, so the PC needs the **gateway's MAC**, not the server's (ARP request = broadcast, reply = unicast).
4. **TCP 3-way handshake** to port 443: **SYN → SYN-ACK → ACK** (1 RTT); the client uses an ephemeral port, e.g. 52814.
5. **TLS handshake**: certificate check, key agreement (TLS 1.3: 1 RTT) → session keys.
6. **HTTP GET** /index.html (encrypted) → segment → packet → frame → bits.
7. **Routing** hop by hop: each router decrements TTL, looks up the destination network, re-frames. tracert / traceroute lists the hops using TTL expiry + ICMP.
8. **Server** decapsulates upward, replies **200 OK** + HTML; the browser then fetches CSS, JS and images on the same connection.
9. **Close** with FIN/ACK, or keep the connection alive for reuse.

Memory trick "**D-A-T-T-H**": DNS, ARP, TCP, TLS, HTTP: the order of first contact.

Exam trap IP **fragments** are reassembled by the destination's **IP** layer; out-of-order **segments** are reordered by **TCP**.

6 HTTP: HyperText Transfer Protocol

high-yield

Definition Application-layer, client–server, request–response protocol of the Web, over TCP port 80 (HTTPS: 443). It is stateless: the server keeps no memory of earlier requests; **cookies** (Set-Cookie → Cookie headers) add state for logins and carts.

Method	Use	Safe	Idem.
GET	read	✓	✓
HEAD	headers only	✓	✓
POST	submit/create	X	X
PUT	replace	X	✓
PATCH	partial edit	X	X
DELETE	remove	X	✓

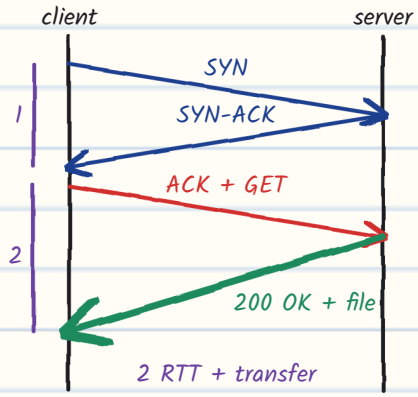
Class	Meaning · examples
1xx	informational · 100, 101
2xx	**success** · 200 OK, 201, 204
3xx	redirect · **301**, 302, 304
4xx	**client error** · 401, 403, **404**
5xx	**server error** · 500, 502, 503

Safe = changes nothing on the server; Idem. (idempotent) = repeating it gives the same result. 401 = not authenticated (log in) · 403 = authenticated but forbidden · 301 = moved permanently · 304 = not modified (use cache) · 503 = service unavailable.

→ Persistent vs non-persistent

Non-persistent (HTTP/1.0 default): a new TCP connection per object, 2 RTT + transfer time each. **Persistent** (HTTP/1.1 default, keep-alive): one connection serves many objects; **pipelining** sends requests back to back.

Worked example ✍ 1 HTML + 10 images, ignore transfer time:
non-persistent, serial: 11 × 2 = **22 RTT**
persistent, no pipelining: 2 + 10 = **12 RTT**
persistent, pipelined: 2 + 1 = **3 RTT**



Version	Year · RFC	What is new
HTTP/1.0	1996 · 1945	headers, status codes; a new connection per object
HTTP/1.1	1997 · 2068	**persistent** by default, pipelining, **Host** header required
HTTP/2	2015 · 7540	**binary framing**, multiplexed streams on one TCP connection, HPACK
HTTP/3	2022 · 9114	over **QUIC on UDP**; no TCP head-of-line blocking; TLS 1.3 built in

Exam trap **HTTPS** is not a separate protocol: it is HTTP inside **TLS** (SSL is its deprecated predecessor) on port 443, giving confidentiality, integrity and server authentication by certificate.

7 The World Wide Web

MCQ favourite

Definition A distributed system of hypertext documents linked by URLs, served over HTTP and shown by a browser. Proposed by **Tim Berners-Lee** at **CERN** in **1989**; first server and browser 1990–91; **Mosaic** (1993) made it popular; **W3C** founded 1994. Three pillars: **HTML** (the page), **URL** (its address), **HTTP** (the transfer). Web servers: Apache, Nginx, IIS.

→ URL anatomy

https://	www.example.com	:443	/notes/osi.html	?lang=en	#layer4
scheme	host (domain)	port (opt.)	path	query	fragment
	www = subdomain, com = TLD			key=value	not sent

scheme://host:port/path?query#fragment

URL = where a resource is; URN = its name (urn:isbn:...); both are **URIs**. The default port comes from the scheme (http 80, https 443). The **#fragment is never sent** to the server.

Aspect	Internet	World Wide Web
What	global network of networks (infrastructure)	one **service** running on the Internet
Made of	routers, links + TCP/IP	pages, browsers, servers + HTTP
Origin	ARPANET, 1969	CERN, 1989
Carries	web, email, FTP, VoIP, SSH ...	web pages only

Memory trick Internet = the **roads**; the Web = **one kind of traffic** on them. Email is another kind on the same roads.

Exam trap Internet ≠ WWW. HTML is a **markup** language, not a programming language. A static page is the same file for all; a dynamic page is built per request (PHP, Node).

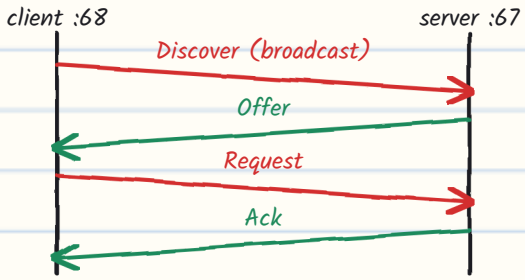
8 FTP, Telnet, SSH

FTP TCP, **two connections**: control on **21** (stays open: "out-of-band" control), data on **20** in active mode (a new connection per file). **Passive** mode: the server names a port and the client connects (firewall friendly). Plain text. **TFTP**: UDP 69, no login.

Telnet vs SSH Both give a remote terminal. **Telnet**: TCP 23, everything, password included, in **plain text**. **SSH**: TCP **22**, encrypted and authenticated (key or password); also carries **SCP/SFTP** and tunnels.

9 DHCP

DHCP Gives a host its IP, mask, gateway and DNS server automatically for a **lease** time. UDP: server **67**, client **68**. Four messages, **DORA**: Discover (broadcast) → Offer → Request (broadcast) → Ack. A **relay agent** forwards them across routers. Successor of BOOTP.



10 Well-Known Ports high-yield

Protocol	Port	Over	Protocol	Port	Over
FTP data / control	20 / 21	TCP	POP3	110	TCP
SSH, SCP, SFTP	22	TCP	NTP	123	UDP
Telnet	23	TCP	IMAP	143	TCP
SMTP (relay)	25	TCP	SNMP / trap	161 / 162	UDP
DNS	53	UDP + TCP	LDAP	389	TCP
DHCP server/client	67 / 68	UDP	HTTPS	443	TCP*
TFTP	69	UDP	SMTP submit	587	TCP
HTTP	80	TCP	IMAPS / POP3S	993 / 995	TCP

* HTTP/3 uses UDP 443. Port = 16 bits → 0–65535: **well-known** 0–1023 · **registered** 1024–49151 · **dynamic / ephemeral** 49152–65535 (IANA). Others: RDP 3389, MySQL 3306, BGP TCP 179.

Memory trick Count up the ladder: **20-21 FTP, 22 SSH, 23 Telnet, 25 SMTP** ("files, secure, plain, mail"). Mail in: POP **110**, IMAP **143**.

Exam trap **UDP**: DNS queries, DHCP, TFTP, SNMP, NTP. **TCP**: FTP, SSH, Telnet, SMTP, POP3, IMAP, HTTP/1.1 and /2. DNS zone transfers use TCP 53.

Worked example A firewall must allow a user to browse securely, fetch mail by IMAP over TLS and log in to a router safely. Open: TCP **443** (and UDP 443 for HTTP/3), TCP **993**, TCP **22**; block Telnet 23 because it sends the password in plain text.

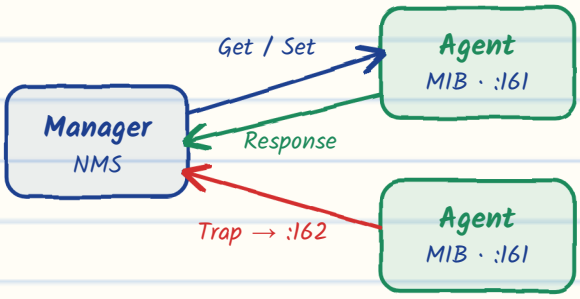
IT link netstat -an lists open ports and connections; nslookup tests DNS; curl -I https://site shows the HTTP status line and headers; ipconfig /renew runs DHCP again.

11 SNMP: Simple Network Management Protocol

high-yield

Definition Application-layer protocol over UDP to **monitor and configure** routers, switches, servers, printers and UPSs. A **manager** polls **agents**; agents can also report events on their own. The agent listens on UDP 161, the manager receives traps on UDP 162.

- **Manager / NMS**: the monitoring station
- **Agent**: software inside each managed device
- **MIB** (Management Information Base): the agent's database of managed objects, e.g. MIB-II
- **OID**: an object's dotted address in a global tree, e.g. sysDescr = 1.3.6.1.2.1.1.1
- **SMI**: rules for naming and typing objects, written in **ASN.1**, encoded with BER



PDU (operation)	Direction	Purpose	Since
GetRequest	manager → agent	read one or more objects	v1
GetNextRequest	manager → agent	read the next OID (walk a table)	v1
SetRequest	manager → agent	change a value (configure)	v1
Response	agent → manager	answer to Get/Set (v1: GetResponse)	v1
Trap	agent → manager	**unsolicited** alert, **not acknowledged**	v1
GetBulkRequest	manager → agent	many objects in one request	**v2**
InformRequest	agent → manager	a notification that **is acknowledged**	**v2**

Version	Security	Notes
SNMPv1 (1990)	**community string**, plain text	defaults "public" (read), "private" (write)
SNMPv2c	still community string	adds GetBulk, Inform, 64-bit counters
SNMPv3	USM: authentication + privacy (encryption); **VACM** access control	the only secure version

Memory trick "Agent **one-six-one** listens; manager **one-six-two** hears the trouble." Polling = the manager asks; a trap = the agent shouts.

Exam trap SNMP runs over **UDP**, not TCP. Encryption arrives only in **v3**; the "c" in v2c means community.

IT link v3 security levels: **noAuthNoPriv** · **authNoPriv** · **authPriv** (authenticated and encrypted). Reading a router's name from a Linux shell: snmpget -v2c -c public 10.0.0.1 1.3.6.1.2.1.1.5.0 (sysName); snmpwalk repeats GetNext down the tree.

Night-Before-the-Exam Sheet

Fact sheet

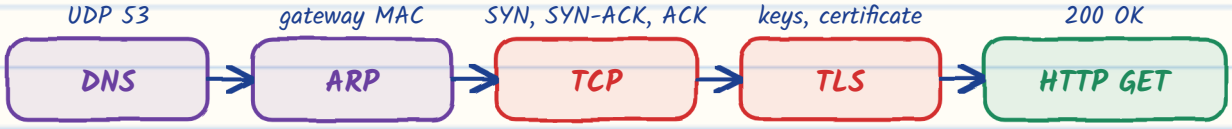
- OSI: ISO 7498, 1984, 7 layers
- PDU: data · segment · packet · frame · bit
- Hub L1 · switch/bridge L2 · router L3
- MAC 48 bit · IPv4 32 · IPv6 128 · port 16
- Encryption, compression → L6
- Routing, logical address → L3
- HTTP 80 · HTTPS 443 · HTTP/3: QUIC
- SNMP UDP 161 agent / 162 trap
- FTP 20/21 · SSH 22 · Telnet 23 · SMTP 25
- TCP/IP: 4 layers (RFC 1122)
- UDP PDU = user datagram
- Gateway: any / all layers
- Only L2 adds a trailer (FCS)
- Dialog control, checkpoints → L5
- Port, end-to-end reliability → L4
- Non-persistent: 2 RTT per object
- DHCP UDP 67/68, DORA
- DNS 53 · POP3 110 · IMAP 143

→ Top 12 one-liners for MCQs

1. WWW: Berners-Lee, CERN, 1989
2. HTTP is stateless; cookies add state
3. HTTP/1.1: persistent by default
4. 404 Not Found = client error (4xx)
5. 301 = moved permanently
6. POST is not idempotent; PUT is
7. MAC changes per hop; IP does not
8. TCP/IP Application = OSI L5 + L6 + L7
9. Trap = unsolicited, unacknowledged
10. SNMPv3 adds auth + encryption
11. FTP control channel is out-of-band
12. URL #fragment is not sent to server

Top traps Physical is layer 1, not 7 · OSI Network = TCP/IP Internet · ARP: MCQ keys usually say Network layer · SSL/TLS: textbook keys say Presentation · Internet ≠ WWW · SNMP and DHCP run over UDP · 401 = unauthenticated, 403 = forbidden · IP reassembles fragments, TCP reorders segments

All tricks together All People Seem To Need Data Processing (7 → 1) · Please Do Not Throw Sausage Pizza Away (1 → 7) · Some People Fear Birthdays (S-P-F-B) · 3-1-1-2 mapping · Russian dolls · D-A-T-T-H · ports 20-21-22-23-25 · agent 161, trap 162



opening https://... in one line

D-A-T-T-H: the first five exchanges of a web visit

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. Which OSI layer is responsible for routing?
(a) Data link (b) Transport
(c) Network (d) Session

2. The PDU of the data link layer is:
(a) frame (b) packet
(c) segment (d) bit

3. Encryption and compression: which layer?
(a) Application (b) Session
(c) Transport (d) Presentation

4. SNMP traps arrive on UDP port:
(a) 161 (b) 162
(c) 143 (d) 110

5. HTTP status 404 belongs to the class:
(a) 2xx (b) 3xx
(c) 5xx (d) 4xx

6. TCP/IP Internet layer = OSI's:
(a) Network (b) Transport
(c) Data link (d) Session

7. Which HTTP method is NOT idempotent?
(a) GET (b) PUT
(c) POST (d) DELETE

8. The FTP control port (TCP) is:
(a) 20 (b) 21
(c) 22 (d) 23

9. Unchanged as a packet crosses routers:
(a) source MAC (b) destination MAC
(c) TTL (d) destination IP

10. HTTP/3 runs over:
(a) QUIC (UDP) (b) TCP only
(c) SCTP (d) raw IP

Answer key → 1-c · 2-a · 3-d · 4-b · 5-d · 6-a · 7-c · 8-b · 9-d · 10-a

Written question With a diagram, explain how data is encapsulated as it moves down the OSI layers when a browser requests a web page, naming the PDU and the header added at each layer. How does decapsulation work at the receiver?

10 marks

1. Draw the 7-layer stacks for client and server with the physical link between them; peer layers talk virtually 2
2. L7–L5: the HTTP request (GET, Host header) is the data; L6 may encrypt it (TLS) 2
3. L4 adds a TCP header (ports 52814 → 443, seq/ack) → **segment**; L3 adds an IP header (IPs, TTL) → **packet**; L2 adds the Ethernet header (MACs, EtherType) + FCS → **frame**; L1 sends **bits** 4
4. Receiver: each layer checks and removes its own header (FCS check, IP address, port → process) and passes the payload up; routers in between work only up to L3 and re-frame 2

→ Before the exam ✓

- ✓ Can list 7 layers with job, PDU, device
- ✓ Can map OSI ↔ TCP/IP and compare them
- ✓ Can trace a URL: DNS → ARP → TCP → TLS → HTTP
- ✓ Know status-code classes and all key ports

Also practise (written):
① Compare OSI and TCP/IP
② Persistent vs non-persistent HTTP with RTT count
③ SNMP architecture and versions
④ FTP's two connections